

Selenium IDE & UI.Vision Advanced Interactions Guide

Practical implementation guide for hovers, drag-and-drop, and keyboard shortcuts

Task 1 Hover over Flipkart 'Categories' Menu (Selenium IDE)

Mouse hover events in Selenium IDE are handled using the `mouse over` command.

Command	Target	Value
<code>open</code>	<code>https://www.flipkart.com</code>	
<code>set window size</code>	<code>1280x800</code>	
<code>mouse over</code>	<code>xpath=//span[contains(text(),'Categories')]</code>	
<code>wait for element visible</code>	<code>css=div._16rR1e</code>	<code>5000</code>
<code>assert element present</code>	<code>css=div._16rR1e</code>	

Task 2 Drag and Drop with UI.Vision RPA Extension

Because standard Selenium IDE often struggles with HTML5 HTML/CSS drag-and-drop APIs, **UI.Vision RPA** utilizes real OS-level mouse coordinates or XClick/XMove commands.

Setup Steps:

1. Install **UI.Vision RPA** from the Chrome Web Store.
2. Open `https://jqueryui.com/droppable/`.
3. Create a new Macro in UI.Vision.

UI.Vision Macro Script:

Command	Target	Value
<code>open</code>	<code>https://jqueryui.com/droppable/</code>	
<code>selectFrame</code>	<code>index=0</code>	
<code>dragAndDropToObject</code>	<code>id=draggable</code>	<code>id=droppable</code>
<code>assertText</code>	<code>css=#droppable p</code>	<code>Dropped!</code>

Alternative (Visual Mode): If HTML drag-and-drop fails, use UI.Vision's visual computer vision commands:
`XClickRelative` or `XMove` with image anchors.

Task 3 Keyboard Shortcuts for Search on Myntra (Selenium IDE)

Selenium IDE simulates keyboard actions using `sendKeys` with specific key variables.

Command	Target	Value
open	https://www.myntra.com	
click	css=input.desktop-searchBar	
type	css=input.desktop-searchBar	shoes
sendKeys	css=input.desktop-searchBar	\${KEY_ENTER}
wait for element visible	css=ul.results-base	5000
assert text	css=h1.title-title	Shoes

Task 4 Limitations of Browser Add-ons for Hover and Drag-and-Drop

1. Synthetic DOM Events vs. Native OS Events:

Browser extensions generate synthetic JavaScript mouse events (`element.dispatchEvent()`). Many modern UI libraries (like React DnD or HTML5 Drag and Drop API) require native browser-level OS input events to trigger state changes, causing tests in extension tools to fail silently.

2. Hover Menu Disappearance (Focus Loss):

When Selenium IDE or DevTools loses window focus or moves to execute the next command step, browser dropdowns created via hover states (`:hover` CSS pseudoclasses or `onmouseover` JS) often disappear instantly before the runner can verify or click child elements.

3. Canvas, iFrames, and Shadow DOM Restrictions:

Drag-and-drop actions across complex canvas elements, nested cross-origin iFrames, or Shadow DOM components cannot be reliably tracked using basic CSS/XPath locators. Extensions lack the low-level API controls provided by full WebDriver frameworks.

Impact on Real-World Apps:

These limitations result in flaky or false-negative test runs in CI/CD pipelines, forcing QA teams to migrate away from browser extensions to robust code-based frameworks (such as Selenium WebDriver in Python/Java, Playwright, or Cypress) that control the browser natively.

Task 5 Navigate Zomato Menu via Hover & Click (Selenium IDE)

Command	Target	Value
open	https://www.zomato.com	
set window size	1280x800	
mouse over	xpath=//p[contains(text(),'Order Online')]	
wait for element visible	xpath=//a[contains(@href,'pizza')]	5000
click	xpath=//a[contains(@href,'pizza')]	
wait for element visible	css=h1	5000
assert text	css=h1	Pizza Delivery